AI Bootcamp Week 3: Model Invocation, Adversarial Agents, and Bounded Agents

Wednesday — Student Exercise Guide

© 2026 Improving

OBJECTIVES

By end of Week 3, the candidate can:

- Explain the DLP automation spectrum and what evidence is required to move up each step
- Explain why the DLP defaults to skills with `disable-model-invocation: true`, and articulate the three graduation criteria for enabling model invocation
- Write invocation test cases (positive trigger, negative trigger, quality) and use them to make a documented enable-or-gap decision on a real skill
- Build an adversarial agent that challenges a skill's output, and explain how it complements (not duplicates) a Python evaluation script
- Run an evaluator-vs-adversarial comparison and use the delta to improve either the EC or the skill
- Compose skills into a bounded agent: a multi-step pipeline run in one IDE session with evaluation and adversarial review at key steps
- Explain context window degradation, why bounded agents hit a ceiling, and what a custom harness is (awareness level; building one is Stage 4 work)
- Describe the IIOS stage framework: where DLP Completion sits (Stage 3, Task) and what Stage 4 (Workflow) requires

© 2026 Improving

TIMING

- **Monday:** 1:00 PM – 5:00 PM (travel morning: 4 hours)
- **Tuesday–Thursday:** 9:00 AM – 5:00 PM with 1-hour lunch (7 hours each)
- **Friday:** 9:00 AM – 12:00 PM (travel afternoon: 3 hours)

© 2026 Improving

WEDNESDAY (9:00 AM – 5:00 PM): ADVERSARIAL AGENTS

9:00 – 10:30 AM: AI CHALLENGES AI

Your Python evaluation scripts measure output against the EC you wrote. That is retrospective quality: does the output meet the defined bar? But what about problems your EC did not anticipate?

An **adversarial agent** is a prompt that receives the output of another skill and attempts to find problems with it: missing requirements, logical errors, edge cases the primary skill missed, quality gaps the evaluation script did not catch.

The complementary pairing:

- **Evaluator:** measures against EC. "Does this output meet the defined quality bar?" Retrospective.
- **Adversarial agent:** challenges without EC. "What is wrong with this output that the EC did not catch?" Proactive.

The evaluator gives you a scorecard. The adversarial agent gives you a red team. Both are needed. A skill that passes its evaluator but fails its adversarial agent has a gap in its EC, which is itself a finding.

Why adversarial agents are AI-native, not code-native: an adversarial agent is a prompt. Every role can write one: product, process, testing, not just developers. This makes proactive quality challenge accessible to the whole team.

Adversarial agents are one form of verification, not the only form. Deterministic checks in your Python evaluation script handle structure: schema validation, field presence, format patterns. Adversarial agents check meaning: is the content actually correct, is the reasoning sound, what did the criteria miss. A complete quality strategy uses both.

10:30 AM – 12:00 PM: THE TWO-PROMPT PROBLEM

When you adopt an adversarial agent, you introduce a second prompt into your quality pipeline:

1. **The worker prompt** (your skill): produces the output. Optimized for producing good work.
2. **The evaluator prompt** (the adversarial agent): checks the output. Optimized for finding problems.

Both can have bugs. Both need evaluation. The mindset for "prompt to check the work" differs from "prompt to do the work": different persona, different EC, different tuning. An adversarial agent that produces false positives is just as harmful as one that misses real issues; it wastes time and erodes trust in the quality process. Common failures: agents that are too permissive ("looks good") or too aggressive (every output has "problems" that are not real).

What makes a good adversarial prompt:

- **Specific role:** "You are a senior QA engineer reviewing this output." Choose the reviewer role that is the natural critic of the skill's output. The best adversarial agent comes from the perspective the skill's author is least likely to have.
- **Clear mission:** "Find as many problems as you can. For each, explain why it is a problem."
- **Full output as input:** the adversarial agent sees everything the primary skill produced.
- **Structured findings:** each problem listed with what is wrong, why it matters, and a recommended fix.

© 2026 Improving

- **Not shown the original EC:** the adversarial agent does not know the criteria. It looks for anything wrong.
- **Calibrated severity:** every finding labeled blocker, major, minor, or nit. An agent that labels everything a blocker needs tuning.
- **Actionable findings:** specific recommendations, not "this is wrong." Vague findings are false positives in disguise.

12:00 – 1:00 PM: LUNCH

1:00 – 3:00 PM: BUILD AND RUN AN ADVERSARIAL AGENT

1. Choose one of your Week 2 SDLC skills: ideally the one you have been evaluating with a Python evaluation script.
2. Write an adversarial prompt for it: specific reviewer role, clear mission, full output as input, structured findings with calibrated severity.
3. Run it against 2–3 real outputs from that skill.
4. For each finding, classify it:
5. Real problem the EC did not cover → gap in your EC. Add a corresponding criterion to your evaluation script.
6. Real problem the EC covers but the skill violated → gap in the skill. Fix the skill.
7. False positive → tune the adversarial prompt.

Commit the adversarial prompt and its findings. The adversarial prompt is an artifact, a reusable quality tool that complements your evaluation script.

3:00 – 5:00 PM: THE EVALUATOR-VS-ADVERSARIAL COMPARISON

Close the loop. Run your evaluation script and your adversarial agent against the same 2–3 outputs. Document the comparison:

- What did the evaluator catch?
- What did the adversarial agent catch that the evaluator missed?
- What did the evaluator catch that the adversarial agent missed?
- What does the delta tell you about gaps in your EC?

Commit the comparison document. Most candidates find their EC cover the obvious quality dimensions but miss domain-specific risks that a well-chosen reviewer role catches. That delta is the value.

The value test: if your adversarial agent only finds what your evaluation script already catches, it is adding nothing. Refine the role, the mission, or the severity calibration until it finds things you did not anticipate.

© 2026 Improving